



Engenharia de software e modelagem II

Prof Me Wesley Soares de Souza

✦
👤 Boas-vindas

Sobre o que iremos falar ?



- **Montando diagramas com visual Paradigm**
 - <https://online.visual-paradigm.com/>
- **Introdução a UML**
- **Diagrama de Classe (parte I)**



Etapas do projeto



1. Refinamento do Modelo de Análise e requisitos

2. Definição da Arquitetura

Método MoSCoW

3. Caso de uso

4. Modelagem de Classes

5. Modelagem de Interações

6. Definição de Interfaces

7. Aplicação de Padrões de Projeto





Home



My Drive



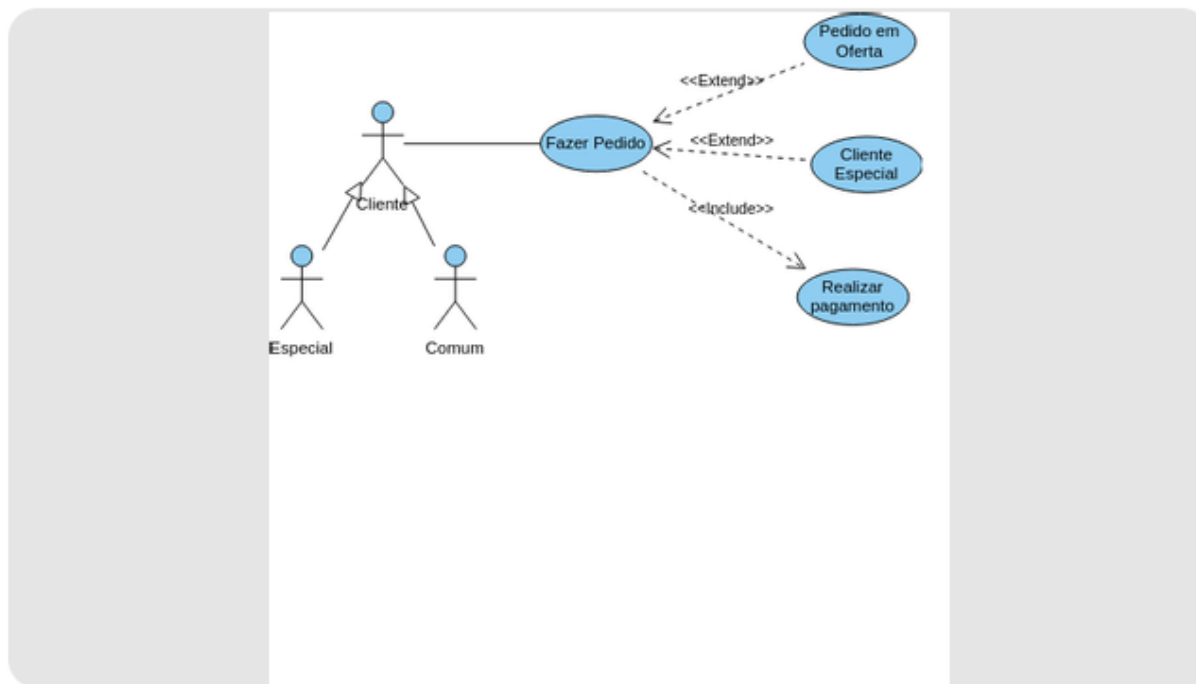
Flipbooks



Slideshows



Videos



Aula FEF



UML – Unified Modeling Language

- Linguagem Gráfica de Modelagem para:
 - Visualizar
 - Especificar
 - Construir
 - Documentar
 - Comunicar Artefatos de sistemas complexos
- Linguagem: vocabulário + regras de combinação



Principal objetivo

- Compreender melhor o sistema que estamos desenvolvendo
- Ter uma visão mais clara do sistema
- Documentar decisões tomadas no projeto
- Especificar comportamento ou a estrutura de um sistema



Princípios básicos

A escolha dos modelos a serem criados tem **profunda influência** sobre a maneira como um determinado problema é atacado e como uma solução é definida

Cada modelo poderá ser expresso em diferentes níveis de precisão



A UML não é !!!!

- um processo
- uma metodologia
- Análise e Projeto OO
- regras de projeto



E o que é ???

- é uma linguagem de notação utilizada para **modelar e documentar sistemas orientados a objetos.**
-

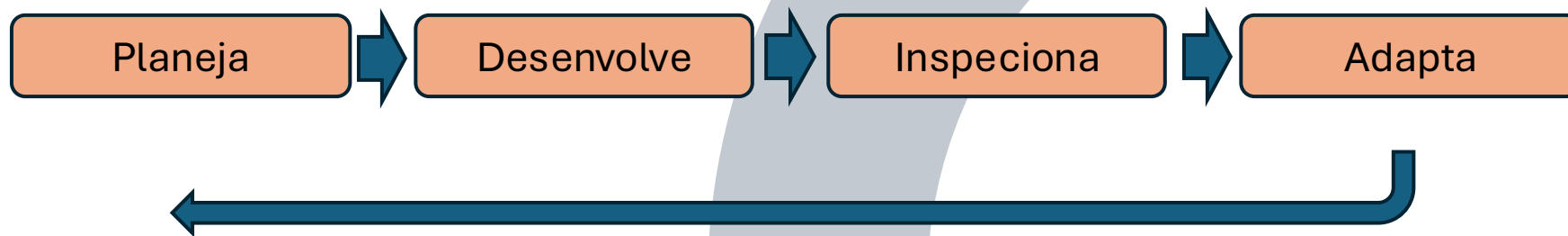
- A UML foi criada por Grady Booch, Ivar Jacobson e James Rumbaugh, combinando métodos que simplificam o design de sistemas.



SCRUM

Um framework para desenvolvimento ágil de produtos

- O **Scrum** é um framework ágil utilizado para desenvolver produtos de forma **iterativa e incremental**, permitindo que a equipe entregue valor continuamente e responda às mudanças.
- O trabalho é dividido em ciclos curtos chamados **Sprints**, nos quais a equipe:



Seguindo em um projeto

Product Backlog

Requisitos do
sistema

Sprint

Seleção de
funcionalidades

Desenvolvimento

Projeto +
implementação +
testes

Review

Apresentação do
resultado

Retrospectiva

O que podemos
melhorar?

SCRUM FRAMEWORK VISUALIZATION

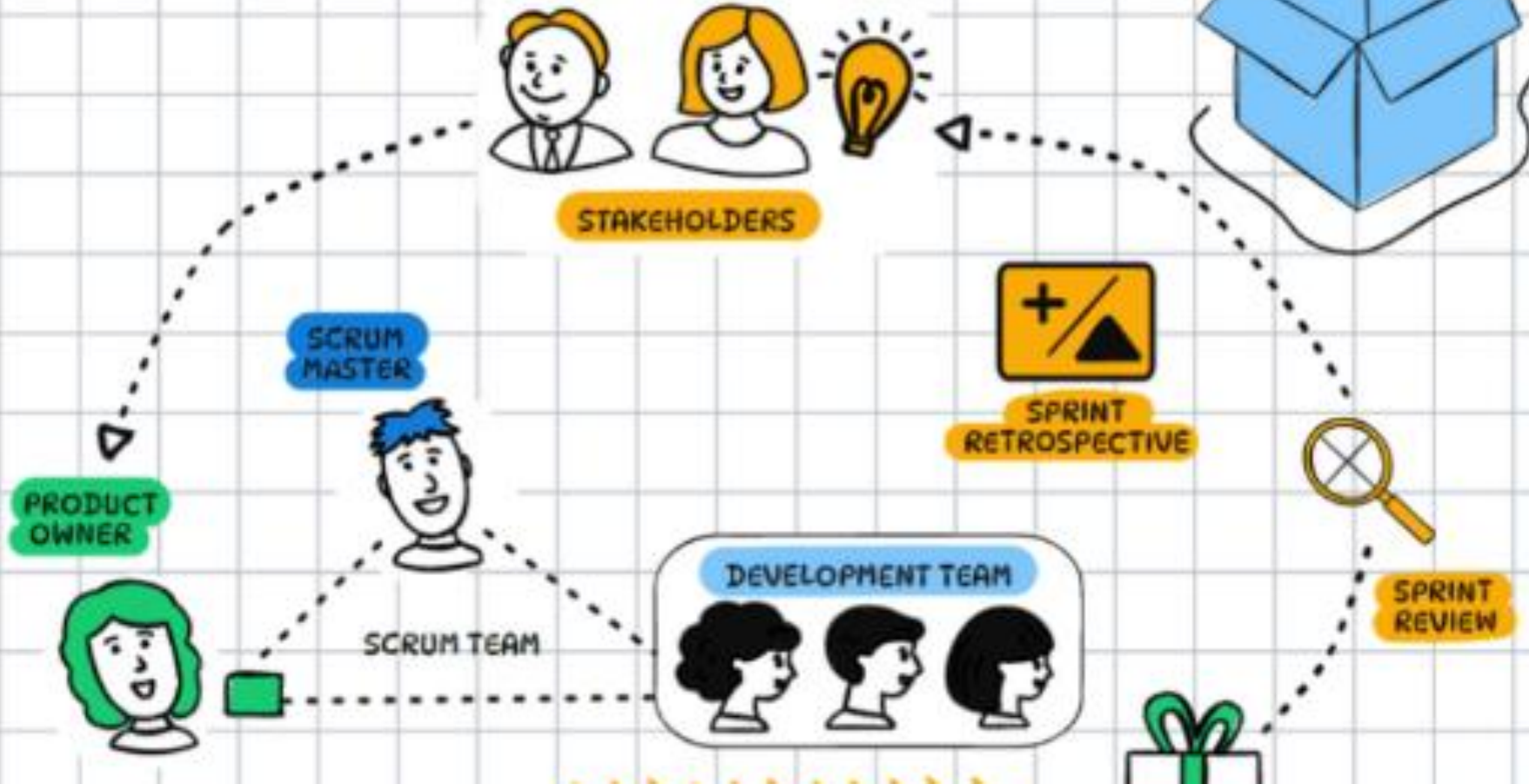
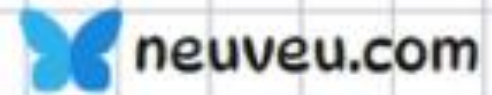




Diagrama de classe

- Mostra um conjunto de classes e seus relacionamentos.
- É o diagrama central da modelagem orientada a objetos.

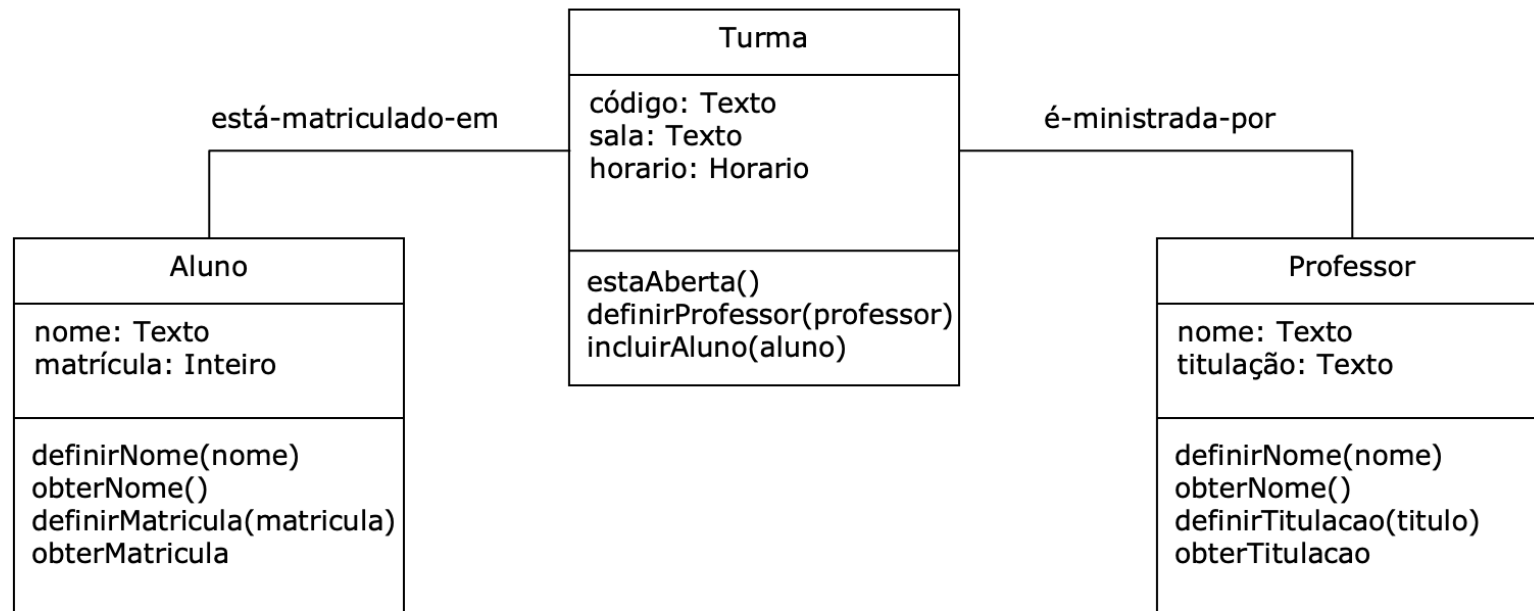


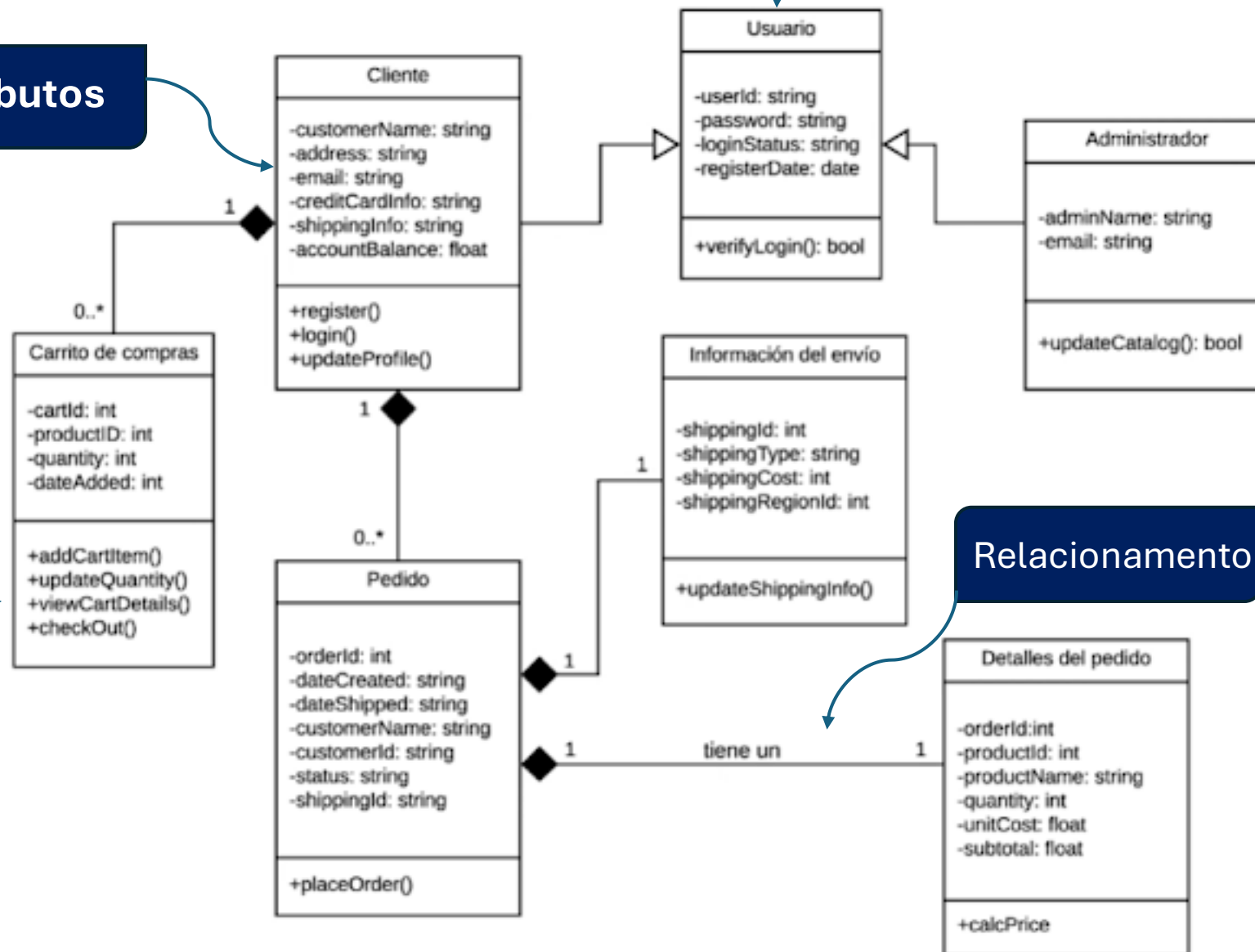
Diagrama de classe

Atributos

Nome

Métodos/ações

Relacionamento



Elementos em um diagrama de classe

- Graficamente, as classes são representadas por retângulos divididos em três partes: o **nome**, **atributos** e **métodos**.



- Devem receber **nomes** de acordo com o problema.
- É comum adotar um padrão para nomes de classes.
- Ex: todos os nomes de classes serão substantivos singulares com a primeira letra maiúscula

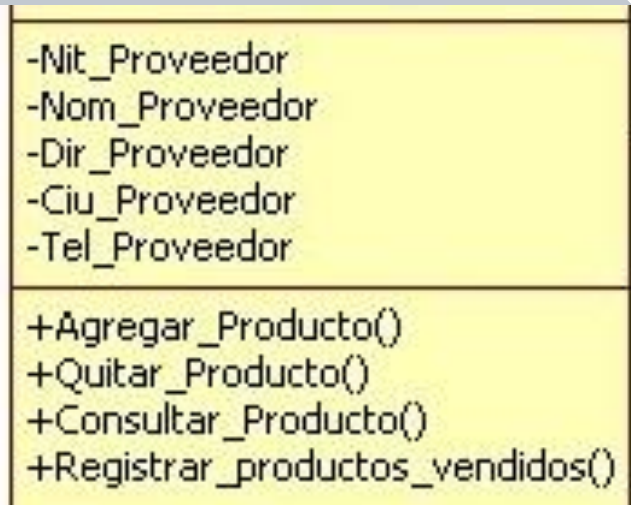
Qual padrão adotar? Isso deve ser uma convenção adotada pelo time ou pela empresa!



Elementos em um diagrama de classe

Atributos:

- Representam o conjunto de características (estado) dos objetos daquela classe
- Visibilidade:
 - **+ público**: visível em qualquer classe de qualquer pacote
 - **# protegido**: visível para classes do mesmo pacote
 - **- privado**: visível somente para classe
- Exemplo:
 - + nome : String



```
classDiagram
    class Class {
        -Nit_Proveedor
        -Nom_Proveedor
        -Dir_Proveedor
        -Ciu_Proveedor
        -Tel_Proveedor
        +Agregar_Producto()
        +Quitar_Producto()
        +Consultar_Producto()
        +Registrar_productos_vendidos()
    }
```

-Nit_Proveedor
-Nom_Proveedor
-Dir_Proveedor
-Ciu_Proveedor
-Tel_Proveedor

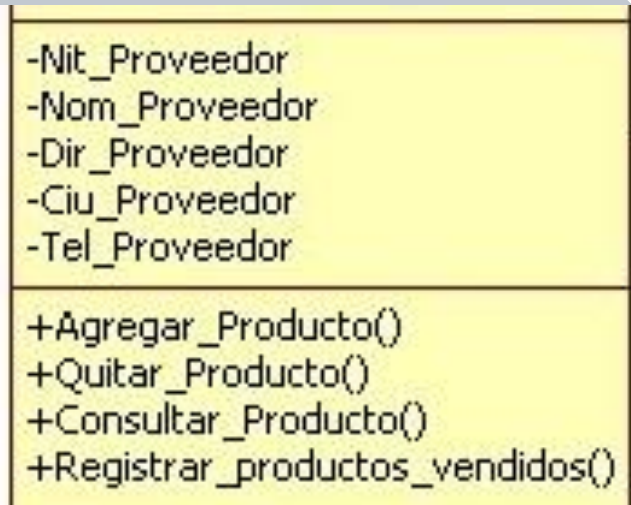
+Agregar_Producto()
+Quitar_Producto()
+Consultar_Producto()
+Registrar_productos_vendidos()



Elementos em um diagrama de classe

Métodos:

- Representam o conjunto de operações (comportamento) que a classe fornece.
- Visibilidade:
 - **+ público**: visível em qualquer classe de qualquer pacote
 - **# protegido**: visível para classes do mesmo pacote
 - **- privado**: visível somente para classe
- Exemplo:
 - `getNome() : String`



```
classDiagram
    class Proveedor {
        -Nit_Proveedor
        -Nom_Proveedor
        -Dir_Proveedor
        -Ciu_Proveedor
        -Tel_Proveedor
        +Agregar_Producto()
        +Quitar_Producto()
        +Consultar_Producto()
        +Registrar_productos_vendidos()
    }
```

-Nit_Proveedor
-Nom_Proveedor
-Dir_Proveedor
-Ciu_Proveedor
-Tel_Proveedor

+Agregar_Producto()
+Quitar_Producto()
+Consultar_Producto()
+Registrar_productos_vendidos()



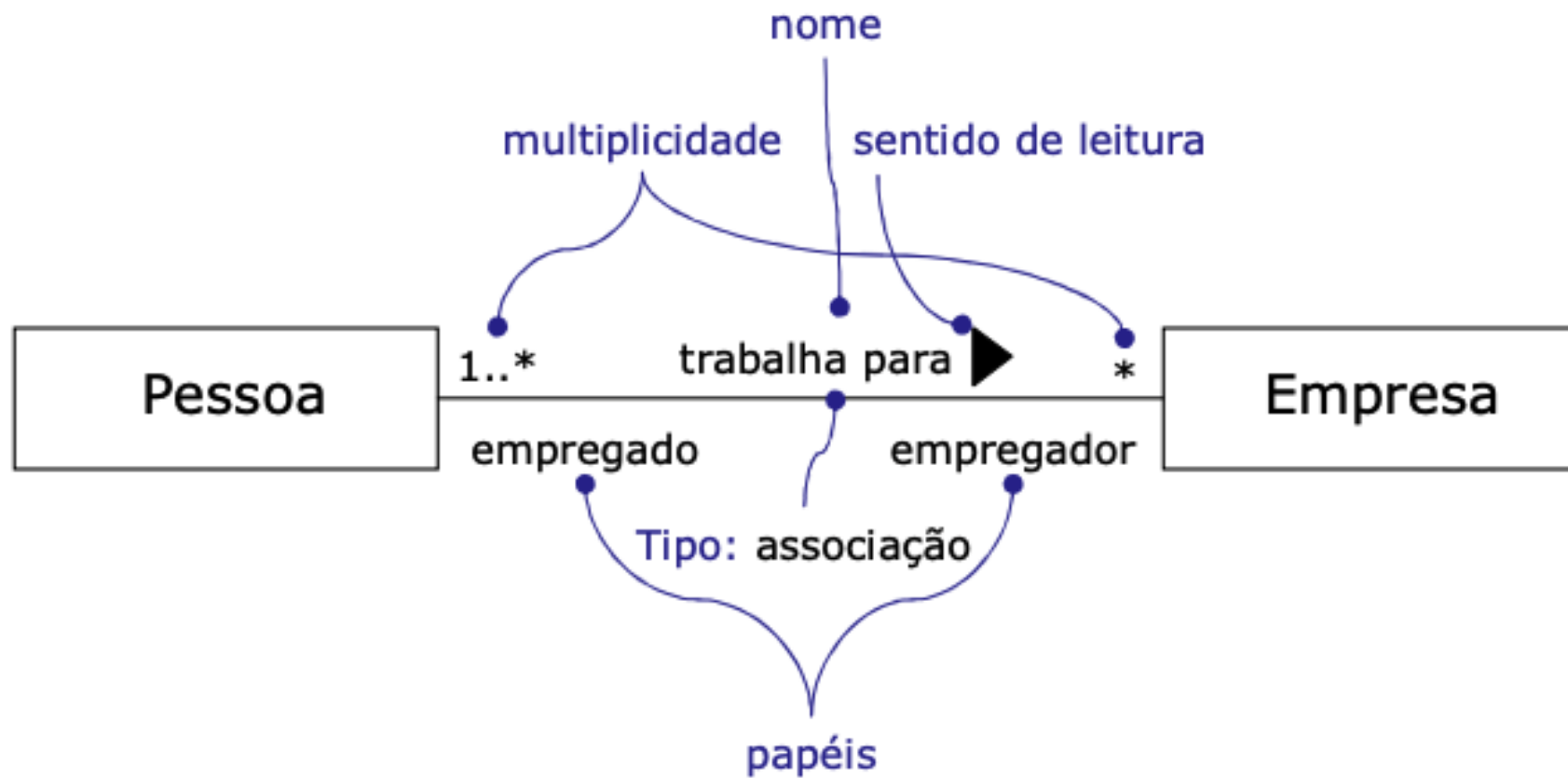
Elementos em um diagrama de classe

Relacionamentos

- Os relacionamentos possuem:
 - **Nome:** descrição dada ao relacionamento (faz, tem, possui,...)
 - **Sentido de leitura**
 - **Navegabilidade:** indicada por uma seta no fim do relacionamento
 - **Multiplicidade:** 0..1, 0..*, 1, 1..*, 2, 3..7
 - **Tipo:** associação (agregação, composição), generalização e dependência
 - **Papéis:** desempenhados por classes em um relacionamento



Relacionamento – Diagrama de classe



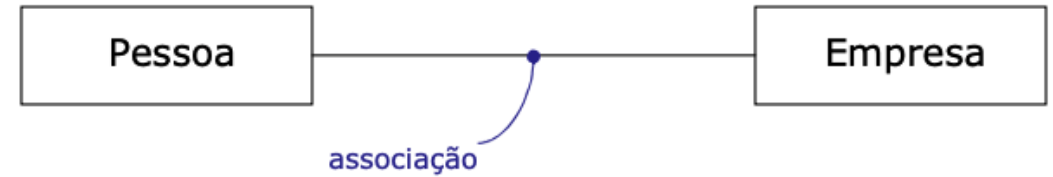
Tipo de relacionamentos

- Indicador de multiplicidade:

- 1 <- Exatamente um
- 1..* <- Um ou mais
- 0..* <- Zero ou mais (muitos)
- * <- Zero ou mais (muitos)
- 0..1 <- Zero ou um
- m..n <- Faixa de valores (por exemplo: 4..7)



Tipo de relacionamentos



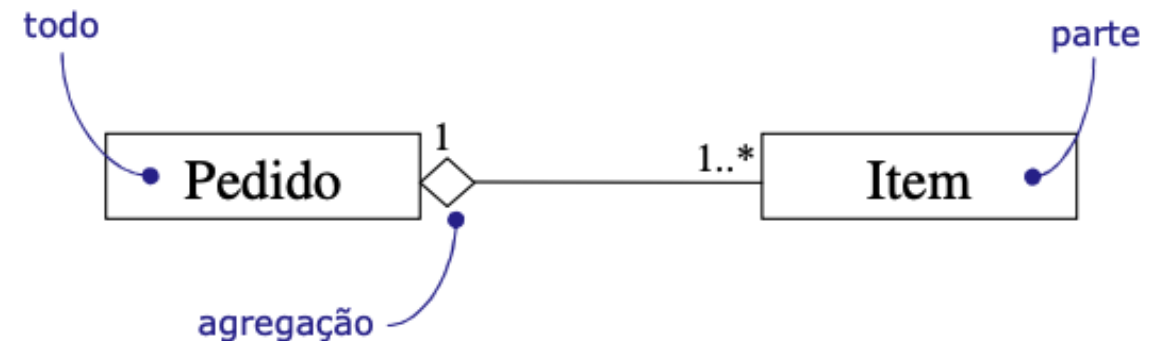
Associação: Indica que **um objeto de uma classe está conectado a um objeto da outra.**

Agregação (◊):

- Representa uma relação

todo–parte fraca.

- A parte pode existir independentemente do todo.
- Exemplo: Pedido ↔ Item (um item pode existir sem um pedido).
- Notação: um **losango vazio** na classe “todo”.



Tipo de relacionamentos



Associa  o

Composi  o (◆)

- Representa uma rela   o **todo-parte forte**.
- A parte n o pode existir sem o todo.
- Exemplo: Casa ◆ — C modo (se a casa   destru  da, os c modos deixam de existir).
- Notac   o: um **losango preenchido** na classe “todo”.



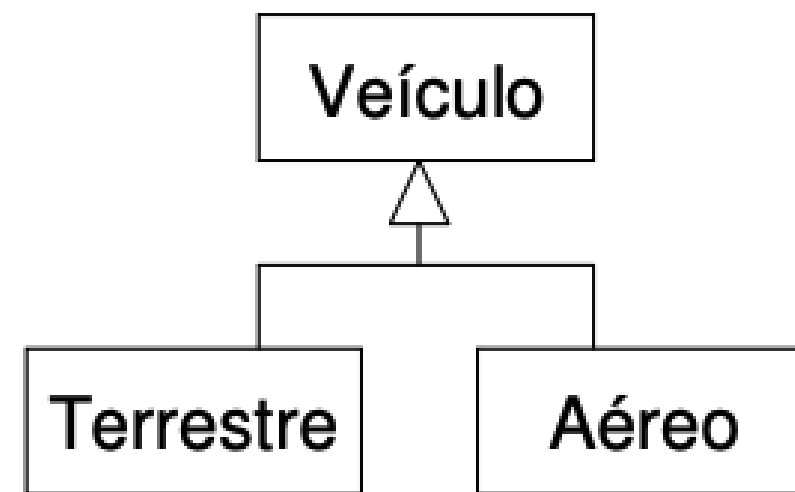
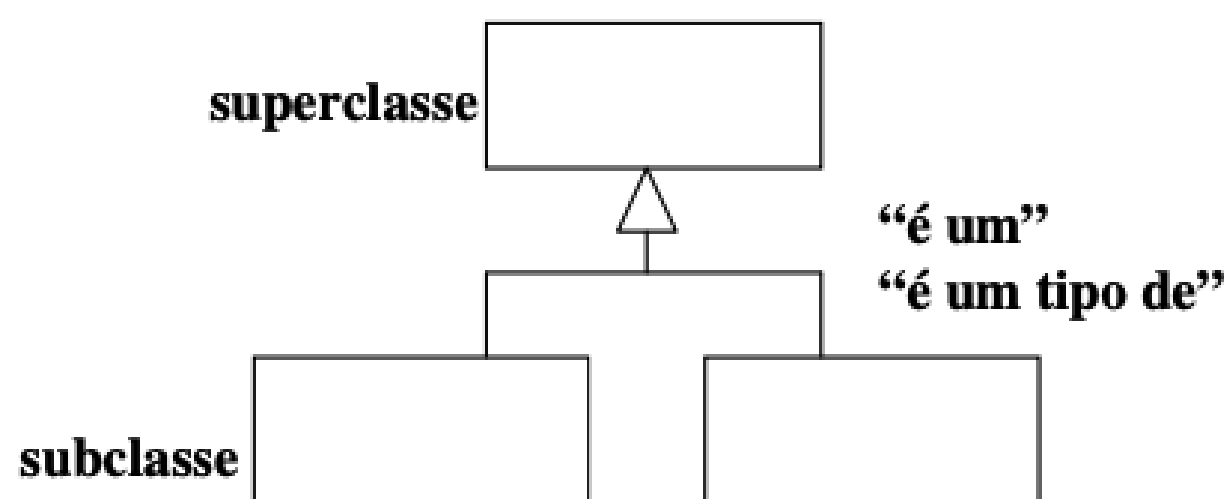
Tipo de relacionamentos

Generalização: Indica que uma classe **especializada herda atributos e comportamentos** de uma classe mais geral.

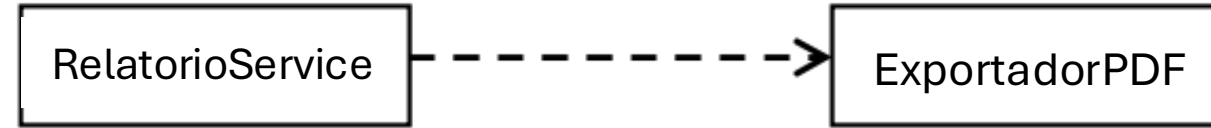
- Exemplo: Funcionario **↑** Gerente, Desenvolvedor.
- Gerente e Desenvolvedor herdam de Funcionario.
- Notação: **seta com triângulo aberto** apontando para a superclasse.



Tipo de relacionamentos



Tipo de relacionamentos



Dependência: Indica que uma classe **usa outra temporariamente**, sem manter uma ligação estrutural permanente.

- É o relacionamento mais fraco.
- Exemplo: RelatorioService usa ExportadorPDF apenas para gerar relatórios, mas não guarda referência permanente.
- Notação: **seta tracejada** com ponta aberta.



🔥 Respiração do Código, Forma 2!

ATÉ A PRÓXIMA!

Guardem suas katanas e fechem seus IDEs! Lembrem-se:

Nenhum bug resiste a um bom refactoring e aos testes automatizados.

🐛 Sem Bugs

🧹 Clean Code

🎓 NIFEF



🦋 "Mantenha o coração queimando e o código compilando!" 🦋